
Causal Detection of Multi-Step LLM Agent Attacks

Viraaji Mothukuri¹ Reza M. Parizi¹

Abstract

Multi-step prompt injection attacks on LLM agents present a fundamental detection challenge because malicious intent emerges only after the workflow completes, while individual actions remain legitimate in isolation. Existing defenses, including input sanitization, output validation, and instruction hierarchy, operate on individual actions or content patterns and cannot capture this sequential structure. We present *CausalTrace*, a detection system that reframes prompt-injection defense as causal inference. It constructs Structural Causal Models from agent trajectories with typed edges capturing data dependency, trust transfer, and state enablement, then applies Pearl’s do-calculus to answer a counterfactual question, namely, whether the harmful outcome would have occurred if the injection had been blocked. This formalization enables a principled distinction between attacks that depend on injections and benign workflows that share surface-level features. Evaluation on a dataset spanning crowdsourced traces, LLM agent benchmarks, and semi-real and real scenarios demonstrates strong detection performance, outperforming content-based baselines while requiring minimal LLM inference cost; bidirectional slicing recovers complete attack chains with high edge recall, providing interpretable explanations that trace exploitation to its causal origins.

1. Introduction

Large language models have enabled autonomous agents that perform complex tasks, including web navigation, API orchestration, and code execution (Yao et al., 2022; Park

et al., 2023; Patil et al., 2024; Qin et al., 2024; Liu et al., 2024b). Unlike traditional machine learning systems, these agents observe environmental state, reason in natural language, and execute multi-step action sequences until accomplishing user objectives (Zhou et al., 2024; Deng et al., 2023). Commercial deployments such as Claude Computer Use (Anthropic, 2024; Ren et al., 2026) and GPT Operator (OpenAI, 2025) demonstrate practical utility, yet this capability introduces fundamental security vulnerabilities that existing defenses inadequately address.

The primary threat is prompt injection, wherein adversarial instructions embedded in external content can hijack agent behavior to perform unauthorized actions (Greshake et al., 2023; Liu et al., 2025). When an agent processes compromised data from web pages, emails, documents, or API responses, embedded instructions may override its original task, enabling credential theft, data exfiltration, or privilege escalation. Indirect prompt injection effectively transforms LLM-integrated applications into attack vectors for data theft and system compromise. As agents are deployed in increasingly sensitive contexts, the security implications of these vulnerabilities become critical.

Existing defenses face an asymmetric challenge. Input sanitization (Chen et al., 2025) and prompt guards (Inan et al., 2023) attempt to filter adversarial content, but attacks can be obfuscated to bypass pattern matching (Liu et al., 2024a; Yu et al., 2024). LLM-based detection methods require expensive inference while achieving limited accuracy (Yuan et al., 2024). Content-based methods have a key limitation because defenders must predict every possible attack phrasing while attackers only need one successful variant.

Security benchmarks measure whether attacks succeed but provide limited insight into how attacks propagate. Agent-Dojo (Debenedetti et al., 2024), InjecAgent (Zhan et al., 2024), WASP (Evtimov et al., 2026), and SafeArena (Tur et al., 2025) record binary outcomes without tracking causal chains from injection to harm. This limitation motivates reframing prompt injection defense as a causal inference problem. The key observation is that attacks become harmful only after workflows are complete, as individual actions remain legitimate in isolation. An agent that reads compromised content, extracts sensitive data, and transmits it to an external endpoint has performed three unremarkable actions.

¹Decentralized Science Lab, College of Computing and Software Engineering, Kennesaw State University, GA 30060, USA. Correspondence to: Viraaji Mothukuri <vmothuku@students.kennesaw.edu>, Reza M. Parizi <rparizi1@kennesaw.edu>.

The attack arises solely from their causal relationship, motivating an analysis of how actions depend on one another rather than of what actions contain.

1.1. Contributions

This work presents *CausalTrace*, a framework for detecting prompt-injection attacks on LLMs via causal graph analysis. The contributions can be summarized as follows.

- (1) A **Causal Graph Formalization** of agent trajectories with typed edges capturing data, trust, and state relationships (§4.1).
- (2) **Formal Causal Inference Detection Methods**, including Counterfactual Analysis, Probabilistic Causal Inference, and Causal Ensemble Detection for principled attack detection. (§4.5).
- (3) **Bidirectional slicing** method for attack chain recovery that identifies injection sources and critical bottlenecks with interpretable explanations. (§4.6).
- (4) **CausalBench**, a benchmark comprising over one million agent trajectories spanning 14 sources, 8 attack categories, and 6 application domains. (§5).

Conflict of Interest Disclosure. The authors disclose that a U.S. Provisional Patent Application related to the methods and technology described in this paper was filed with the U.S. Patent and Trademark Office on October 31, 2025, Application No. 63/909,130.

2. Related Work

This section surveys existing research on prompt injection attacks and defenses, security benchmarks for LLM agents, detection approaches for agent trajectories, and causal inference techniques in security applications.

Prompt injection attacks exploit language models’ inability to distinguish between instructions and data. Direct injection manipulates user prompts through carefully crafted adversarial inputs (Perez & Ribeiro, 2022; Liu et al., 2025), while indirect injection embeds malicious instructions in external sources that agents process during task execution (Zhan et al., 2024). Attack techniques have grown increasingly sophisticated, with gradient-based optimization generating adversarial suffixes (Zou et al., 2023), AutoDAN automating jailbreak generation through hierarchical search (Liu et al., 2024a), and GPTFuzz applying mutation-based fuzzing to discover novel attack vectors (Yu et al., 2024). These advances demonstrate the fundamental difficulty of content-based defense approaches.

Agent-specific vulnerabilities extend beyond standalone LLM attacks. Analysis of agent architectures reveals 2-3×

higher vulnerability rates compared to standalone models, attributable to goals embedded in system prompts, multi-step action generation that amplifies reasoning errors, and observational interfaces that expand the attack surface (Chiang et al., 2025). Multi-agent systems introduce additional vectors including orchestrator coercion, memory poisoning, and cyclic agent interactions (Trail of Bits, 2025; Friedman et al., 2025). Visual perception further expands the attack surface, with pixel-level perturbations capable of steering vision-language model-based agents (Wang et al., 2025).

Existing **defenses** employ several strategies with varying effectiveness. Input sanitization filters adversarial content through structured formatting (Chen et al., 2025) or classifier-based detection (Meta AI, 2024), but struggles with semantic variation and obfuscation techniques. Instruction hierarchy training teaches models to prioritize system prompts over external content, though this provides limited protection against indirect injection where malicious instructions appear in legitimate data sources. Output validation approaches require defining valid action boundaries for open-ended tasks (Debenedetti et al., 2024), which proves impractical for general-purpose agents.

Architectural defenses provide stronger guarantees at a higher cost. CaMeL interposes a capability-aware layer that extracts trusted programs from user queries and prevents untrusted inputs from altering control flow (Debenedetti et al., 2025), achieving substantial attack prevention but requiring abandonment of standard frameworks. Deception-based approaches deploy honeytokens and trap payloads to detect malicious agents during reconnaissance (Ayzenshteyn et al., 2025). LLM-based detection employs auxiliary models to classify trajectories as safe or harmful (Yuan et al., 2024), though with limited accuracy and expensive inference requirements. None of these approaches explicitly models the causal structure that enables multi-step attacks.

Several **benchmarks** evaluate agent security across different dimensions. AgentDojo provides 629 test cases across realistic tasks, including email management, banking, and travel booking (Debenedetti et al., 2024). InjecAgent covers 1,054 scenarios targeting tool-integrated agents (Zhan et al., 2024). WASP evaluates prompt injection under realistic constraints, finding high initiation rates but limited end-to-end goal completion (Evtimov et al., 2026). SafeArena measures compliance with harmful tasks across the categories of misinformation, illicit activities, and cybercrime (Tur et al., 2025). R-Judge benchmarks LLM safety judges on trajectory-level risk assessment (Yuan et al., 2024). These benchmarks measure binary attack outcomes without tracking causal dependencies from injection to harm.

Causal inference provides formal tools for reasoning about cause and effect beyond statistical correlation (Pearl, 2009; 2019). Structural Causal Models (SCM) consist of variables,

structural equations that define causal mechanisms, and directed acyclic graphs that encode dependencies. The key capability is counterfactual reasoning, determining what would have happened under different conditions through do-calculus interventions (Pearl, 2018). Prior security applications include intrusion detection through audit log analysis, where causal graphs track how system events enable subsequent compromise (King & Chen, 2003; Pohly et al., 2012), and malware analysis, where causal dependencies between API calls reveal malicious behavior patterns (Liu et al., 2018). Dynamic taint analysis tracks untrusted data through program execution to detect information flow violations (Clause et al., 2007; Newsome & Song, 2005). These applications share the insight that attacks exhibit a causal structure distinct from benign behavior, and that this structure is more robust than surface features.

Provenance and dependency graph analysis applies graph-theoretic methods to security monitoring. Provenance systems such as PASS (Muniswamy-Reddy et al., 2006) and SPADE (Gehani & Tariq, 2012) construct dependency graphs via system call interposition for forensic reconstruction. Information-flow frameworks, including Flume (Krohn et al., 2007) and JFlow (Myers, 1999), enforce taint propagation policies to prevent unauthorized disclosure. Attack investigation systems such as SLEUTH (Hossain et al., 2017) and Holmes (Milajerdi et al., 2019) apply dependency tracking for real-time APT detection. These approaches rely on *reachability-based* detection, flagging attacks when tainted data reaches sensitive sinks through any valid path. CausalTrace differs by distinguishing data, trust, and state edge semantics for cross-domain violation detection; employing watermark provenance orthogonal to taint for identifying untrusted influence without explicit data flow; and applying counterfactual inference to evaluate whether outcomes persist absent injection sources rather than relying solely on path existence.

CausalTrace applies these insights to LLM agent security, modeling trajectories as causal graphs where edges represent data dependencies, trust transfers, and state enablements. Unlike existing detection methods that analyze action content or temporal sequences, causal analysis identifies whether injection sources are connected to sensitive sinks through the agent’s observation-reasoning-action flow.

3. Problem Setting

This section formalizes the agent execution model, threat model, and attack definition. We then introduce the causal graph representation that underlies our detection approach and describe the benchmark that instantiates these definitions.

3.1. Agent Execution Model

An LLM-powered agent receives a user-specified task q and executes a sequence of actions $\mathbf{a} = (a_1, \dots, a_T)$ to accomplish the objective. Each action a_t is characterized by a type drawn from `{navigate, click, type, read, tool_call}`, action-specific parameters, an origin domain, and parent action identifiers capturing structural causality. The agent observes state s_t after each action and generates the subsequent action according to $a_{t+1} \sim \pi_\theta(\cdot \mid q, \mathbf{a}_{1:t}, s_t)$. This observation-reasoning-action loop continues until the task is completed or fails.

The parent field in each action captures the agent’s natural execution structure without requiring explicit data flow annotations. Navigation events spawn DOM observations, observations feed LLM calls, LLM calls produce actions, and actions trigger new navigations. This structural causality exists in any agent system and provides the foundation for causal analysis.

3.2. Threat Model

The adversary embeds adversarial instructions into external content that agents encounter during task execution, such as web pages, documents, email bodies, and API responses. The attacker cannot modify the user’s task specification, system prompt, or agent code, nor can they observe the agent’s internal reasoning. Successful attacks cause the agent to perform actions that serve the attacker’s objectives rather than the user’s goals, including data exfiltration, credential theft, unauthorized transactions, privilege escalation, and malicious code execution.

We consider adaptive adversaries employing sophisticated techniques beyond static payload placement. These include cloaking strategies that serve different content to agents versus humans, behavioral triggers that inject payloads only after specific conditions are met, and multi-visit activation patterns that serve benign content initially and payloads on subsequent visits. The adversary may control content on multiple public websites and coordinate attacks across them, planting content on one site designed to influence agent behavior when the agent later visits a different trusted site. We additionally consider evasion-aware adversaries who craft payloads designed to avoid detection while achieving their objectives. Such adversaries employ obfuscation techniques, distribute malicious instructions across multiple observations, or structure attacks to mimic benign execution patterns. We additionally consider evasion-aware attackers who observe CausalTrace’s detection criteria. Such adversaries face a fundamental constraint, as achieving harmful outcomes *requires* causal paths from attacker-controlled content to sensitive actions. Obfuscating content evades pattern matching but preserves taint propagation. Mimicking benign topology requires the attack to actually behave be-

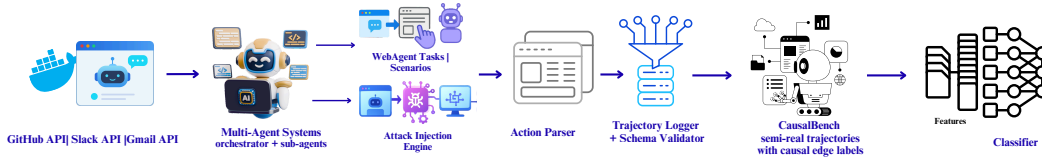


Figure 1. CausalBench generation pipeline.

nignly. Distributing payloads across sessions defeats single-trajectory analysis but leaves cross-session memory edges detectable through persistent watermarking.

3.3. Causal Graph Representation

We represent agent trajectories as directed acyclic graphs $G = (V, E)$ where nodes V correspond to actions and edges E capture causal dependencies between them. Each edge (v_i, v_j, t) has a type t indicating the nature of the causal relationship. Data dependency edges connect actions where a_j consumes data produced or observed by a_i , such as extracting an email address and typing it into a form. Trust transfer edges connect actions where a_j executes instructions encountered in a_i 's observation, representing the critical boundary where external content influences agent behavior. State enablement edges connect actions in which a_i establishes preconditions required by a_j , such as authentication that enables access to protected resources.

Each node carries a trust level $\ell \in \{\text{trusted}, \text{untrusted}, \text{sensitive}\}$ derived from action type and content. User-initiated actions and system prompt instructions are trusted sources. Observations from external content, including web pages, emails, and API responses, are untrusted sources. Actions involving credential access, financial transactions, code execution, and data transmission to external endpoints are sensitive sinks. This labeling requires no manual annotation and is computed automatically from the trajectory.

3.4. Attack Definition

An attack trajectory contains a causal path from an injection source to a sensitive sink. Formally, trajectory τ is classified as an attack if there exist nodes v_i, v_j in the causal graph such that v_i carries an untrusted or injection-like label, v_j is a sensitive sink, and a directed path exists from v_i to v_j in the causal graph. The attack chain comprises all nodes on paths from injection sources to sensitive sinks, recoverable via bidirectional traversal (§4.6). Benign trajectories may contain both injection-like observations from legitimate but oddly worded pages and sensitive actions from normal user tasks, but these nodes are not causally connected in the

graph structure.

This formulation captures the essential property distinguishing attacks from benign behavior. Multi-step attacks require malicious content from an external source to causally influence a dangerous action through the agent's observation-reasoning-action flow. Detection reduces to graph reachability augmented with counterfactual reasoning about whether the sensitive action would occur absent the injection.

3.5. CausalBench

The preceding definitions require a benchmark that provides agent trajectories matching the execution model (§3), contains attacks conforming to the threat model, represents trajectories as causal graphs with typed edges, and labels attack chains per the formal attack definition. Figure 1 illustrates the generation pipeline. Multi-agent systems comprising an orchestrator and task-specific sub-agents execute against live service APIs (GitHub, Slack, Gmail) running in Docker containers, preserving authentic tool behavior. An attack injection engine embeds adversarial payloads across 8 categories at three sophistication levels into task executions. Trajectories are parsed into structured events following the event schema, validated for DAG consistency and label coherence, and annotated with causal edges linking injection sources to sensitive sinks. The resulting benchmark provides controlled attack and benign trajectories with ground-truth edge labels for training and evaluation.

4. Method

CausalTrace detects prompt injection attacks via an ensemble pipeline that combines LLM-based semantic analysis, taint propagation, integrity watermarking, and causal mechanism integrity testing. The core insight is that injection attacks hijack the causal mechanism generating agent actions: the intended mechanism $A = f(T, S_{\text{trusted}})$ where actions depend on user task and trusted state becomes $A = g(T, S_{\text{trusted}}, I)$ where injected content I influences behavior. Figure 2 illustrates the CausalTrace pipeline, where causal graphs are analyzed through parallel taint propagation, watermark verification, and a causal analysis layer

comprising counterfactual, probabilistic, and mechanism integrity tests. The Attack Detection Classifier is trained on trajectories generated through the CausalBench pipeline (Figure 1).

4.1. Causal Graph Construction

Given an agent trajectory $\tau = (a_1, \dots, a_T)$, we construct an SCM $\mathcal{M} = (V, E, F)$ where nodes V represent actions, edges E represent causal dependencies, and F defines structural equations specifying how each action depends on its causal parents. We infer three edge types capturing distinct causal relationships. Data dependency edges connect actions where a_j consumes data produced by a_i , such as extracting an email address and typing it into a form. Trust transfer edges connect actions in which a_j executes instructions based on observation o_i , representing the critical boundary where external content influences agent behavior. State enablement edges connect actions in which a_i establishes preconditions for a_j , such as authentication that enables access to protected resources.

The key distinction underlying mechanism integrity is between external content providing DATA versus INSTRUCTIONS. Legitimate external content informs agent decisions: a webpage that provides product prices, which the agent incorporates into a purchase decision. Injection provides directives that hijack behavior: a webpage containing override instructions targeting attacker-controlled endpoints. The LLM component classifies this distinction through semantic analysis, while taint propagation and causal analysis verify whether classified injections influence sensitive actions.

4.2. Taint Propagation and Integrity Watermarking

We adapt dynamic taint analysis from program security to agent trajectories, tracking how untrusted content influences subsequent actions. Cross-domain data dependencies from untrusted sources to sensitive sinks constitute a structural signature of prompt injection, regardless of content obfuscation. Complementing taint analysis, we embed provenance markers to detect tampering with trusted execution lineage. While taint tracks contamination from untrusted sources, watermarking tracks preservation of trusted provenance, providing independent detection signals.

Each node receives initial labels based on action type and origin. User-initiated actions and system prompt instructions receive trusted labels with watermark tokens and empty taint sets. Observations from external sources receive untrusted labels with taint sets containing themselves. Actions involving credential access, code execution, and data transmission are considered sensitive sinks and require taint-free lineage. Taint propagates transitively through data dependency and trust transfer edges in topological order; state enablement edges do not propagate taint because they represent capa-

bility rather than data flow. A taint violation occurs when tainted data reaches a sensitive sink; a watermark gap occurs when a sensitive action lacks watermark lineage.

4.3. Probabilistic Causal Inference

We formalize attack detection using Pearl’s causal hierarchy. Let I denote injection presence and M denote malicious outcome. Level 1 (association) asks whether I and M co-occur. Level 2 (intervention) asks what happens to M under $\text{do}(I)$. Level 3 (counterfactual) asks whether M would have occurred if I were blocked, given both were observed. The framework operates at Levels 2 and 3, computing the Average Treatment Effect via graph mutilation and the Probability of Necessity via abduction, action, and prediction.

We instantiate an SCM where each action node follows the structural equation below. The binary variable $I \in \{0, 1\}$ indicates whether the node contains injection content. The variable $s \in [0, 1]$ denotes injection strength, reflecting the degree of adversarial influence present in the content. The variable $T \in [0, 1]$ represents task influence propagated from parent nodes through legitimate execution. The effect parameters β_I and β_T govern the relative contribution of injection and task signals, respectively.

$$P(M \mid \text{pa}(M)) = \underbrace{\beta_I \cdot I \cdot s}_{\text{injection effect}} + \underbrace{\beta_T \cdot T \cdot (1 - I)}_{\text{task effect}} \quad (1)$$

Parameters calibrated from multi-benchmark data. $\beta_I = 0.75$ and $\beta_T = 0.05$ reflect weighted average attack success rates from WASP (72%), BrowserART (66%), pajamas (90%), and AgentDojo (68%), consistent with reported rates of 58-90% in prompt injection literature. The Average Treatment Effect measures causal strength: $\text{ATE} = P(M \mid \text{do}(I=1)) - P(M \mid \text{do}(I=0))$. The do-operator implements graph mutilation by removing incoming edges to I , setting I to the specified value, and propagating effects through structural equations.

The Probability of Necessity answers whether the attack would NOT have occurred without injection: $\text{PN} = P(M_0=0 \mid I=1, M=1)$. Computing PN requires three steps. Abduction infers noise terms U consistent with observations by inverting structural equations: $U = \text{clip}(\text{observed} - f(\text{pa}, 0), 0, 1)$. Action creates the counterfactual world via graph mutilation setting $\text{do}(I=0)$. Prediction forward-propagates through the mutilated DAG in topological order, applying structural equations with inferred noise terms. Key assumptions include additive invertible noise, correctly specified structural equations, a uniform prior $\text{Beta}(1, 1)$ over exogenous factors, and the Markov condition excluding hidden confounders. Sensitivity analysis in the supplementary material demonstrates robustness

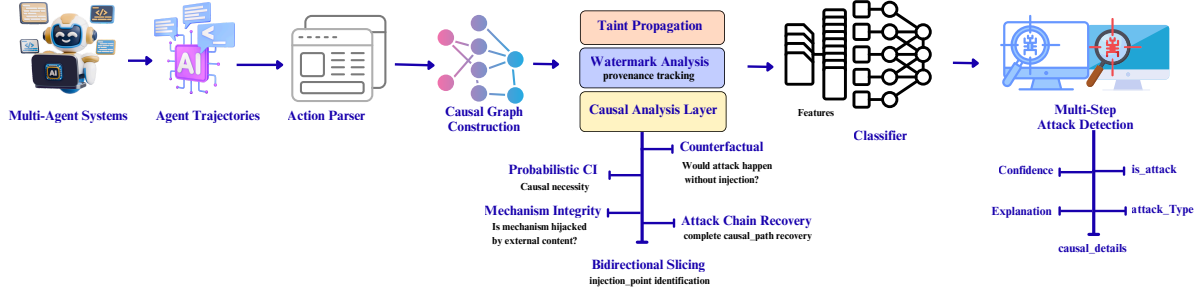


Figure 2. Overview of CausalTrace Detection Pipeline

across parameter ranges.

A high PN alone does not distinguish malicious injection from user-authorized cross-domain actions. When a user explicitly requests the agent to follow external instructions (e.g., "install nginx using documentation at this URL"), removing that content would also prevent task completion, yielding $PN \approx 1.0$ despite the action being legitimate. To resolve this ambiguity, we condition the counterfactual on the user task specification τ through a task alignment score $\alpha(q, M) \in [0, 1]$ measuring the overlap between the user task and the flagged outcome. When $PN > \theta$ and α is high, the outcome aligns with the user's stated objective and the trajectory is classified as approved rather than malicious. In deployment, α can be instantiated as a policy-based allowlist of approved external sources and action types, providing deterministic disambiguation without additional inference cost.

4.4. Ensemble Classification

While individual detection signals each capture aspects of attack behavior, no single detector reliably identifies all attack variants. Single-step injections with explicit override language differ fundamentally from multi-step attacks, where individual actions appear benign. CausalTrace employs an LLM-first architecture where semantic classification determines the detection pathway: for single-step attacks, the LLM provides immediate classification; for multi-step attacks, it triggers the full causal analysis pipeline.

The ensemble combines five complementary detectors targeting distinct attack signatures. The LLM Semantic Classifier analyzes content for injection patterns through natural language understanding. The Graph Neural Network learns structural patterns using GCN, GAT, or GIN architectures on the causal graph representation. The Random Forest operates on the feature vector capturing chain depth, cross-domain ratios, and taint patterns. The Mechanism Integrity detector tests d-separation of sensitive actions from untrusted sources. Probabilistic Causal Inference estimates PN, flagging cases where outcomes depend causally on in-

jection. Each detector produces a binary classification, and the ensemble returns an attack label when a configurable majority threshold is exceeded.

4.5. Detection

Algorithm 1 presents the complete detection procedure integrating all components. CausalTrace employs a multi-stage architecture that combines LLM-based routing with causal analysis for efficient and accurate attack detection. The LLM serves as an initial router, classifying whether the trajectory contains injection and whether multi-step causal analysis is warranted. Only high-confidence single-step classifications exit early (lines 3–8). Obfuscated, uncertain, or multi-step cases proceed to Stage 2, where the algorithm constructs the causal graph, propagates taint and provenance markers, and computes causal metrics using Pearl's do-calculus. Edge types are inferred through rule-based heuristics on node metadata (same-domain sequential events as data dependency, cross-domain observation-to-action as trust transfer, state-modifying actions as state enablement) rather than through LLM classification.

4.6. Attack Chain Recovery

We adapt bidirectional slicing from program analysis to recover complete attack chains from the injection source to the harmful outcome. The key insight is that attacks constitute a causal subgraph: the intersection of nodes influenced by malicious content and nodes necessary for harmful outcomes. Backward slicing from a malicious outcome v_M captures all causally necessary predecessors through reverse traversal: $\mathcal{B}(v_M) = \{v \in V : v \rightsquigarrow v_M\}$. This identifies preconditions and intermediate steps required for attack completion, corresponding to the program slice in traditional analysis.

Forward slicing from an injection node v_I captures all potentially contaminated actions: $\mathcal{F}(v_I) = \{v \in V : v_I \rightsquigarrow v\}$. This bounds the scope of malicious influence regardless of whether harm ultimately occurred, corresponding to the impact set. The critical attack chain emerges from the slice

Algorithm 1 CAUSALTRACE

Require: Trajectory τ , policy π , thresholds θ
Ensure: Attack decision, confidence, intervention

```

1: // Stage 1: semantic classification
2:  $(is\_inj, is\_multi, conf) \leftarrow \text{LLMCLASSIFY}(\tau)$ 
3: if  $is\_inj \wedge \neg is\_multi \wedge conf > \theta_{single}$  then
4:   return (attack, conf, BLOCK)
5: end if
6: if  $\neg is\_inj \wedge conf > \theta_{benign}$  then
7:   return (benign, conf, ALLOW)
8: end if
9: // Stage 2: Causal graph construction (Multi-step cases fall through to causal analysis)
10:  $G \leftarrow \text{BUILDGRAPH}(\tau)$ 
11:  $taint, wm \leftarrow \text{PROPAGATETAINTWATERMARK}(G)$ 
12:  $tv \leftarrow \exists v : taint[v] \neq \emptyset \wedge \text{ISSINK}(v)$ 
13:  $wg \leftarrow \exists v : wm[v] = \emptyset \wedge \text{ISSINK}(v)$ 
14: // Stage 3: Probabilistic Causal Inference (Pearl Level 2-3)
15:  $I \leftarrow \text{IDENTIFYINJECTIONNODES}(G)$ 
16:  $M \leftarrow \text{IDENTIFYMALICIOUSNODES}(G)$ 
17:  $G_{do} \leftarrow G \setminus \text{INCOMINGEDGES}(I)$  {Graph mutilation}
18:  $P_1 \leftarrow P(M=1 \mid G)$ ;  $P_0 \leftarrow P(M=1 \mid G_{do}, I=0)$ 
19:  $ATE \leftarrow P_1 - P_0$  {Level 2: Intervention}
20:  $U \leftarrow \text{ABDUCT}(G, I=1, M=1)$  {Infer noise terms}
21:  $M' \leftarrow \text{PROPAGATE}(G_{do}, U, I=0)$  {Counterfactual prediction}
22:  $PN \leftarrow 1 - P(M'=1)$  {Level 3: Necessity}
23: // Stage 4: Ensemble voting
24:  $\mathbf{d} \leftarrow [is\_inj, \text{GNN}(G), \text{RF}(G), tv \vee wg, PN > \theta_{pn}]$ 
25:  $is\_attack \leftarrow \text{SUM}(\mathbf{d}) \geq \theta_{maj}$ 
26: // Stage 5: Intervention decision
27: if  $is\_attack \wedge (\exists v : v.dom \in \pi_{block})$  then
28:   return (attack,  $\text{SUM}(\mathbf{d})/|\mathbf{d}|$ , BLOCK)
29: else if  $is\_attack$  then
30:   return (attack,  $\text{SUM}(\mathbf{d})/|\mathbf{d}|$ , WARN)
31: end if
32: return (benign,  $1 - \text{SUM}(\mathbf{d})/|\mathbf{d}|$ , ALLOW)

```

intersection:

$$\mathcal{C}(v_I, v_M) = \mathcal{F}(v_I) \cap \mathcal{B}(v_M) \quad (2)$$

This formulation isolates nodes both influenced by injection and necessary for the attack, excluding benign preamble actions and collateral effects that do not contribute to the attack.

Bottleneck detection identifies critical intervention points within the recovered chain. A bottleneck $v_b \in \mathcal{C}$ satisfies the property that all paths from v_I to v_M traverse v_b , representing a high-value target where blocking a single action prevents attack completion. The framework ranks bottlenecks by causal depth, prioritizing early intervention points that prevent maximal downstream harm. Both slicing operations require a single-pass graph traversal in topological order, enabling chain recovery with minimal computational overhead.

5. Evaluation and Results

5.1. Experimental Setup

The evaluation extends CausalBench (§3.5) with large-scale synthetic and real-world trajectories to form a benchmark of over one million agent trajectories from 14 sources, spanning 143 service domains across 11 industry verticals. To achieve large-scale coverage of tool-calling behaviors, the benchmark incorporates 1,010,394 trajectories from the Toucan dataset (Xu et al., 2025) with attack trajectories synthesized at three sophistication levels. To validate generalization across established evaluation frameworks, the benchmark incorporates trajectories from established agent benchmarks, including Mind2Web (Deng et al., 2023), ToolEmu (Ruan et al., 2024), MARBLE (Zhu et al., 2025), AgentBench (Liu et al., 2024b), and BrowserGym (de Chezelles et al., 2025). The complete implementation and dataset are available at the GitHub (Viraaji Mothukuri, 2026) repository for reproducibility.

We compare against six baselines spanning commercial and open-source defenses. PromptGuard (Meta AI, 2024) is Meta’s prompt injection classifier trained on adversarial prompts. NeMo Guardrails (Rebedea et al., 2023) is NVIDIA’s content safety filter for conversational AI. LLM-Guard (Protect AI, 2024) and Rebuff (Protect AI, 2023) are open-source prompt injection detectors that employ ML-based classification. KeywordFilter (Greshake et al., 2023) applies regular expression patterns to match known malicious indicators. ActionWhitelist (Liu et al., 2024c) flags actions outside a predefined allowed set. Additionally, we compare against graph-based classifiers, including GCN (Kipf & Welling, 2017) and GAT (Veličković et al., 2018), trained on the same causal trajectory graphs.

5.2. Results

We evaluate CausalTrace on detection accuracy, generalization across domains, component contributions, and attack chain recovery. Figure 3 shows the distribution of attack categories in our evaluation corpus, with data exfiltration (17.5%) and credential theft (15.0%) comprising the most prevalent categories. Figure 4 presents corpus composition integrating semi-real trajectories with real-world benchmarks.

Performance remains consistent across application domains. Figure 5 shows above 97% F1 across all domains, with DevOps achieving 99.6%, Shopping 99.5%, Email 99.4%, and Social 99.3%. The heterogeneous Other category shows 97.0% F1 due to diverse trajectory patterns not well-represented in training data.

Systematic ablation quantifies component contributions on the semi-real validation set. Figure 6 (left) shows that the full system achieves 99.2% F1, while removing watermark

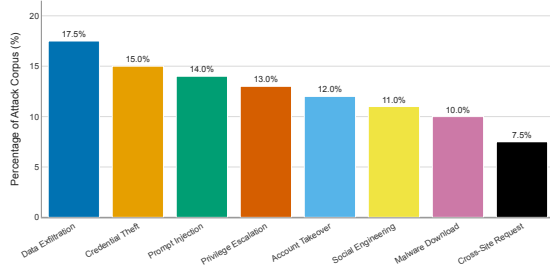


Figure 3. Distribution of attack categories in the evaluation corpus.

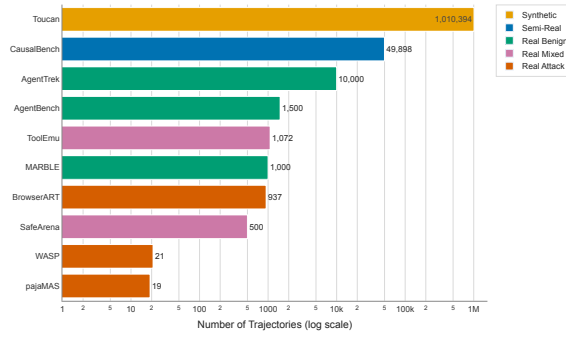


Figure 4. Composition of the evaluation corpus by data source.

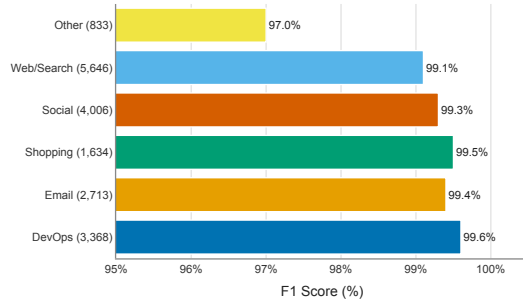


Figure 5. Per-domain detection performance with trajectory counts.

and taint features reduces F1 to 99.1% and 99.0%, respectively. Causal graph topology alone achieves 99.1% F1, though taint and watermark features provide defense-in-depth against adaptive adversaries. Figure 6 (right) shows standalone detection rates for individual ensemble components. The Attack Detection Classifier achieves 99.1% F1 when operating on the 34 causal graph features produced by the upstream pipeline (SCM construction, edge typing, taint propagation, and watermark verification). Counterfactual achieves 84.2% F1, and Probabilistic CI achieves 73.7% as individual detectors within the ensemble. These

components are not independent alternatives; the Attack Detection Classifier depends on the causal graph that the other components also operate on.

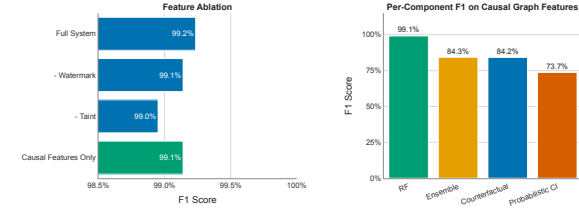


Figure 6. Ablation study showing feature contributions (left) and method comparison (right).

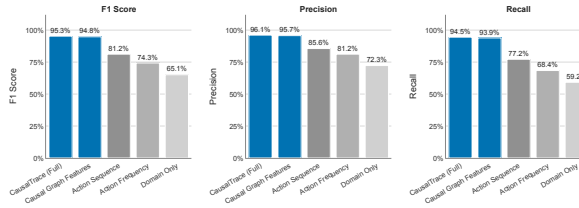


Figure 7. Comparison of causal graph methods against non-causal baselines.

Figure 7 evaluates on a held-out benchmark comprising real attack trajectories from InjecAgent, WASP, and pajaMAS. Methods leveraging the causal graph achieve 95.3% F1 with 96.1% precision and 94.5% recall, while baselines relying on action sequences (81.2%), action frequencies (74.3%), or domain features (65.1%) exhibit degraded performance.

On the 1,477-trajectory out-of-distribution (OOD) evaluation comprising real-world benchmarks unseen during training, CausalTrace achieves 92.6% F1. Per-benchmark recall is 85.8% (BrowserART), 86.8% (SafeArena), 100% (WASP), and 84.2% (pajaMAS). On the combined evaluation (CausalBench and OOD), F1 is 98.0% with a bootstrap 95% confidence interval of [97.7%, 98.3%] over 10,000 resamples. The 6.8 percentage point gap between in-distribution (99.4%) and OOD (92.6%) reflects a genuine domain shift from semi-real training to real-world benchmarks. Performance remains above 84% recall across all four OOD sources, indicating that learned causal patterns transfer to unseen attack distributions. False positive analysis confirms low misclassification on benign trajectories. On 250 OOD benign trajectories from SafeArena, zero false positives were observed. On the CausalBench evaluation, 3 false positives occurred across 27,398 benign trajectories (FPR = 0.011%), all involving legitimate cross-domain API

calls where taint propagation patterns coincidentally match attack signatures.

Sensitivity analysis confirms that detection performance is not brittle to causal parameter calibration. OOD F1 remains at 92.6% across all β_I and β_T perturbations tested, indicating that the upstream causal graph features carry the primary detection signal independently of the probabilistic causal module’s parameterization. To quantify dependence on inferred edge structure, auxiliary (non-backbone) edges were corrupted at increasing rates while preserving the sequential event ordering. F1 remains flat through 30% corruption (0.926 to 0.924, -0.2%), degrades proportionally through 60% (0.875, -5.5%), and collapses only when nearly all auxiliary edges are removed (100% corruption yields F1 of 0.394, -57.4%). The sequential backbone comprises 36% of all edges, so 100% auxiliary corruption removes 64% of the total edge set. These results confirm that the model is robust to moderate noise in inferred causal dependencies and that auxiliary causal edges carry essential discriminative signal beyond sequential ordering alone.

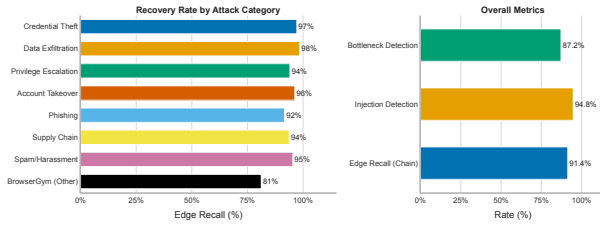


Figure 8. Attack chain recovery performance by category (left) and aggregate metrics (right).

Figure 8 presents chain recovery performance. CausalTrace achieves strong edge recall on well-structured attack types, including Data Exfiltration (98%), Credential Theft (97%), and Account Takeover (96%). The BrowserGym (Other) category exhibits 81% due to heterogeneous formats, where 51% of trajectories lack explicit causal edge annotations. In such cases, a temporal fallback heuristic reconstructs chains using action-sequence ordering. Aggregate metrics show 91.4% edge recall, 94.8% injection detection, and 87.2% bottleneck identification with 0.15ms slice computation time. The causal graph analysis stage operates with sub-millisecond latency and requires no LLM inference, as taint propagation, watermark verification, and feature extraction are computed deterministically from graph structure. The initial routing stage uses an LLM semantic classifier, which can be replaced by a distilled open-source model for deployment without proprietary API dependencies.

6. Limitations

CausalTrace assumes access to complete agent trajectories and is designed for post-hoc audit of logged sessions rather than real-time interception. The probabilistic causal module assumes additive noise, correctly specified structural equations, and the Markov condition excluding hidden confounders. Causal parameter estimation (β_I, β_T) is based on benchmark attack rates and may require recalibration in novel domains. Chain recovery varies with annotation quality and degrades on heterogeneous trajectory sources that lack explicit causal edge annotations. Attacks that influence reasoning through factual-seeming content without cross-domain trust-transfer or taint signals remain challenging, as do structure-aware adversaries who mimic benign causal topology, though achieving harmful outcomes still requires causal paths from attacker-controlled content to sensitive actions.

7. Conclusion

Detecting prompt injection in LLM agents fundamentally differs from defending standalone language models. Attacks unfold across multi-step trajectories where malicious instructions embedded in observations propagate through reasoning to produce harmful actions. Content-based defenses cannot capture this structure, achieving substantially lower F1 scores on our evaluation corpus. CausalTrace reframes detection as causal inference over agent execution graphs. The key insight is that attacks exhibit distinctive causal signatures. Formalizing trajectories as SCM with edges capturing data dependency, trust transfer, and state enablement enables detection through do-calculus interventions that establish counterfactual dependence between injection sources and harmful outcomes. Beyond binary classification, causal analysis provides interpretable forensics. Bidirectional slicing recovers complete attack chains, bottleneck identification pinpoints critical intervention points, and counterfactual reasoning quantifies the necessity of specific injection content. These capabilities address a gap in current agent security tooling, where detection without explanation provides limited actionable insight. Several directions merit further investigation. Extending causal analysis to streaming trajectories would enable real-time intervention before attack completion. Formal robustness guarantees against structure-aware adversaries remain an open theoretical challenge. Transfer learning across agent architectures could reduce domain-specific calibration requirements. As agents assume greater autonomy in high-stakes domains, understanding the causal mechanics of compromise becomes not merely useful but essential.

Impact Statement

CausalTrace addresses a security challenge for LLM-powered agents deployed in high-stakes domains, including financial services, healthcare, and enterprise automation. By detecting prompt injection through causal analysis rather than surface-level content patterns, the framework supports post-hoc audit and forensic analysis of agent behavior with interpretable attack explanations. Its high detection accuracy and low overhead for graph-based analysis make it practical in settings where repeated LLM-based detection would be costly.

At the same time, detailed characterization of attack causal structures could help adversaries better understand how multi-step attacks unfold. However, the constraints identified by CausalTrace arise from the mechanics of prompt-injection attacks rather than from implementation-specific weaknesses. Sharing of specific attack patterns should therefore follow responsible disclosure practices. Because trajectory logging may capture sensitive information about user tasks, deployments should include appropriate access controls, retention policies, and privacy safeguards. CausalTrace should be used as part of a defense-in-depth strategy alongside preventive measures such as input sanitization, instruction hierarchy training, and output validation.

References

- Anthropic. Claude 3 model card: Introducing computer use, a new claude 3.5 sonnet, and a new claude 3.5 haiku. 2024. URL <https://www.anthropic.com/news/3-5-models-and-computer-use>.
- Ayzenshteyn, D., Weiss, R., and Mirsky, Y. Cloak, honey, trap: Proactive defenses against LLM agents. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 8095–8114, Seattle, WA, August 2025. USENIX Association. ISBN 978-1-939133-52-6. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/ayzenshteyn>.
- Chen, S., Piet, J., Sitawarin, C., and Wagner, D. Struq: defending against prompt injection with structured queries. In *Proceedings of the 34th USENIX Conference on Security Symposium, SEC '25, USA*, 2025. USENIX Association. ISBN 978-1-939133-52-6.
- Chiang, J. Y. F., Lee, S., Huang, J.-B., Huang, F., and Chen, Y. Harmful helper: Perform malicious tasks? web AI agents might help. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. URL <https://openreview.net/forum?id=4KoMbO2RJ9>.
- Clause, J., Li, W., and Orso, A. Dytan: a generic dynamic taint analysis framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis, ISSTA '07*, pp. 196–206, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595937346. doi: 10.1145/1273463.1273490. URL <https://doi.org/10.1145/1273463.1273490>.
- de Chezelles, T. L. S., Gasse, M., Lacoste, A., Caccia, M., Drouin, A., Boisvert, L., Thakkar, M., Marty, T., Assouel, R., Shayegan, S. O., Jang, L. K., Lù, X. H., Yoran, O., Kong, D., Xu, F. F., Reddy, S., Neubig, G., Cappart, Q., Salakhutdinov, R., and Chapados, N. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=5298fKGmv3>.
- Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=m1YYAQjO3w>.
- Debenedetti, E., Shumailov, I., Fan, T., et al. Defeating prompt injections by design. *arXiv:2503.18813*, 2025. URL <https://arxiv.org/abs/2503.18813>.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL <https://openreview.net/forum?id=kiYqbO3wqw>.
- Evtimov, I., Zharmagambetov, A., Grattafiori, A., Guo, C., and Chaudhuri, K. WASP: Benchmarking web agent security against prompt injection attacks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL <https://openreview.net/forum?id=Ip1lCUAllL>.
- Gehani, A. and Tariq, D. Spade: support for provenance auditing in distributed environments. In *Proceedings of the ACM/IFIP/USENIX International Middleware Conference, Middleware '12*, pp. 101–120, Berlin, Heidelberg, 2012. Springer-Verlag. ISBN 9783642351693.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec*

- '23, pp. 79–90, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702600. doi: 10.1145/3605764.3623985. URL <https://doi.org/10.1145/3605764.3623985>.
- Hossain, M. N., Milajerdi, S. M., Wang, J., Eshete, B., Gjomemo, R., Sekar, R., Stoller, S., and Venkatakrishnan, V. SLEUTH: Real-time attack scenario reconstruction from COTS audit data. In *26th USENIX Security Symposium (USENIX Security 17)*, pp. 487–504, Vancouver, BC, August 2017. USENIX Association. ISBN 978-1-931971-40-9. URL <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hossain>.
- Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D., and Khabsa, M. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023. URL <https://arxiv.org/abs/2312.06674>.
- King, S. T. and Chen, P. M. Backtracking intrusions. *SIGOPS Oper. Syst. Rev.*, 37(5):223–236, October 2003. ISSN 0163-5980. doi: 10.1145/1165389.945467. URL <https://doi.org/10.1145/1165389.945467>.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL <http://openreview.net/forum?id=SJU4ayYgl>.
- Krohn, M., Yip, A., Brodsky, M., Cliffer, N., Kaashoek, M. F., Kohler, E., and Morris, R. Information flow control for standard os abstractions. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles, SOSOP '07*, pp. 321–334, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595935915. doi: 10.1145/1294261.1294293. URL <https://doi.org/10.1145/1294261.1294293>.
- Liu, X., Xu, N., Chen, M., and Xiao, C. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=7Jwpw4qKkb>.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., and Tang, J. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Liu, Y., Zhang, M., Li, D., Jee, K., Li, Z., Wu, Z., Rhee, J. J., and Mittal, P. Towards a timely causality analysis for enterprise security. In *Network and Distributed System Security Symposium*, 2018. URL <https://api.semanticscholar.org/CorpusID:51951640>.
- Liu, Y., Jia, Y., Geng, R., Jia, J., and Gong, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 1831–1847, Philadelphia, PA, August 2024c. USENIX Association. ISBN 978-1-939133-44-1. URL <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yup ei>.
- Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., Zhang, L. Y., and Liu, Y. Prompt injection attack against llm-integrated applications, 2025. URL <https://arxiv.org/abs/2306.05499>.
- Meta AI. Prompt guard: A classifier for detecting prompt injections and jailbreaks. <https://huggingface.co/meta-llama/Prompt-Guard-86M>, 2024.
- Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., and Venkatakrishnan, V. Holmes: Real-time apt detection through correlation of suspicious information flows. In *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 1137–1152, 2019. doi: 10.1109/SP.2019.00026.
- Muniswamy-Reddy, K.-K., Holland, D. A., Braun, U., and Seltzer, M. Provenance-aware storage systems. In *Proceedings of the Annual Conference on USENIX '06 Annual Technical Conference, ATEC '06*, pp. 4, USA, 2006. USENIX Association.
- Myers, A. C. Jflow: practical mostly-static information flow control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '99*, pp. 228–241, New York, NY, USA, 1999. Association for Computing Machinery. ISBN 1581130953. doi: 10.1145/292540.292561. URL <https://doi.org/10.1145/292540.292561>.
- Newsome, J. and Song, D. X. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In *Network and Distributed System Security Symposium*, 2005. URL <https://api.semanticscholar.org/CorpusID:99191>.
- OpenAI. Operator system card. 2025. URL <https://openai.com/operator>.
- Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive

- simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701320. doi: 10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>.
- Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive apis. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 126544–126565. Curran Associates, Inc., 2024. doi: 10.52202/079017-4020. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/e4c61f578ff07830f5c37378dd3ecb0d-Paper-Conference.pdf.
- Pearl, J. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009. URL https://archive.illc.uva.nl/cil/uploaded_files/inlineitem/Pearl_2009_Causality.pdf.
- Pearl, J. Theoretical impediments to machine learning with seven sparks from the causal revolution. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, WSDM '18, pp. 3, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450355810. doi: 10.1145/3159652.3176182. URL <https://doi.org/10.1145/3159652.3176182>.
- Pearl, J. The seven tools of causal inference, with reflections on machine learning. *Commun. ACM*, 62(3):54–60, February 2019. ISSN 0001-0782. doi: 10.1145/3241036. URL <https://doi.org/10.1145/3241036>.
- Perez, F. and Ribeiro, I. S. Ignore previous prompt: Attack techniques for language models. *ArXiv*, abs/2211.09527, 2022. URL <https://api.semanticscholar.org/CorpusID:253581710>.
- Pohly, D. J., McLaughlin, S., McDaniel, P., and Butler, K. Hi-fi: collecting high-fidelity whole-system provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*, ACSAC '12, pp. 259–268, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450313124. doi: 10.1145/2420950.2420989. URL <https://doi.org/10.1145/2420950.2420989>.
- Protect AI. Rebuff: LLM prompt injection detector. <https://github.com/protectai/rebuff>, 2023. URL <https://github.com/protectai/rebuff>.
- Protect AI. LLM Guard: The security toolkit for LLM interactions. <https://github.com/protectai/llm-guard>, 2024. URL <https://github.com/protectai/llm-guard>.
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., dahai li, Liu, Z., and Sun, M. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., and Cohen, J. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In Feng, Y. and Lefever, E. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 431–445, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-demo.40. URL <https://aclanthology.org/2023.emnlp-demo.40/>.
- Ren, X., Jiang, P., Li, K., Huang, Z., Du, X., Jiang, J., Xing, Z., Sun, J., and Zhuo, T. Y. Hackworld: Evaluating computer-use agents on exploiting web application vulnerabilities. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=nLfZPoJb07>.
- Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GEcwtMklua>.
- Trail of Bits. pajamas: Practical agentic jailbreak attacks on multi-agent systems. 2025. URL <https://blog.trailofbits.com/2025/07/31/hijacking-multi-agent-systems-in-your-pajamas/>.
- Triedman, H., Jha, R. D., and Shmatikov, V. Multi-agent systems execute arbitrary malicious code. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=DAozI4etUp>.
- Tur, A. D., Meade, N., Lù, X. H., Zambrano, A., Patel, A., DURMUS, E., Gella, S., Stanczak, K., and Reddy, S. Safearena: Evaluating the safety of autonomous web agents. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=7TrOBcxSvy>.

- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJXMpikCZ>.
- Viraaji Mothukuri, R. M. P. Causal trace github repository, 2026. URL <https://github.com/decentralizedsciencelab/CausalTrace.git>.
- Wang, X. et al. Webinject: Prompt injection attack to web agents. In *EMNLP 2025*, 2025. URL <https://aclanthology.org/2025.emnlp-main.104/>.
- Xu, Z., Soria, A. M., Tan, S., Roy, A., Agrawal, A. S., Poovendran, R., and Panda, R. Toucan: Synthesizing 1.5m tool-agent data from real-world mcp environments, 2025. URL <https://arxiv.org/abs/2510.01179>.
- Yao, S., Zhao, J., Yu, D., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022. URL <https://openreview.net/forum?id=tvI4ulylcqs>.
- Yu, J., Lin, X., Yu, Z., and Xing, X. Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts, 2024. URL <https://arxiv.org/abs/2309.10253>.
- Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., Wang, R., and Liu, G. R-judge: Benchmarking safety risk awareness for LLM agents. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 1467–1490, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.79. URL <https://aclanthology.org/2024.findings-emnlp.79/>.
- Zhan, Q., Liang, Z., Ying, Z., and Kang, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL <https://aclanthology.org/2024.findings-acl.624/>.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oKn9c6ytLx>.
- Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, Z., Wang, Z., Qian, C., Tang, X., Ji, H., and You, J. MultiAgentBench : Evaluating the collaboration and competition of LLM agents. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8580–8622, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.421. URL <https://aclanthology.org/2025.acl-long.421/>.
- Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models, 2023. URL <https://arxiv.org/abs/2307.15043>.

A. Graph Representations

CausalTrace employs three graph representations with increasing semantic richness. A generic graph $G = (V, E)$ provides structural connectivity but lacks semantic constraints on nodes or edges. Agent trajectory graphs extend this formalism by introducing typed nodes and labeled edges.

$$G_{\text{agent}} = (V, E, \tau, \lambda) \quad (3)$$

The function $\phi : V \rightarrow \{\text{Task, Observation, Action, Result}\}$ assigns node types and $\lambda : E \rightarrow \{\text{temporal, data, causal}\}$ assigns edge labels. Table 1 and Table 2 specify the semantics of each type.

Table 1. Node Type Specification

Type	Symbol	Description
Task	T	Initial instruction provided to the agent
Observation	O	Information perceived from environment
Action	A	Operation executed by agent
Result	R	Outcome of execution

Table 2. Edge Type Specification

Type	Notation	Semantics
Temporal	e_t	Sequential ordering in execution
Data	e_d	Information flow between nodes
Causal	e_c	Cause-effect relationship

Causal graphs constitute the most expressive representation, augmenting the agent trajectory graph with structural equations and an injection variable.

$$G_{\text{causal}} = (V, E, \mathcal{F}) \quad (4)$$

The set $\mathcal{F} = \{f_v : v \in V\}$ specifies structural equations determining each variable from its parents. For attack detection, the node set is augmented with an injection variable I representing adversarial content. The causal structure enables counterfactual queries via Pearl’s do-calculus, formalized through the Probability of Necessity.

$$\text{PN} = P(M_{I \leftarrow 0} = 0 \mid I = 1, M = 1) \quad (5)$$

The term $M_{I \leftarrow 0}$ denotes the counterfactual outcome under intervention $do(I = 0)$. High PN indicates the injection was a necessary cause of the malicious outcome. Table 3 summarizes the distinguishing characteristics of each representation.

Table 3. Comparison of Graph Representations

Property	Generic	Agent Trajectory	Causal
Node typing	None	$\phi : V \rightarrow \mathcal{T}$	$\tau + \text{Injection}$
Edge typing	None	$\lambda : E \rightarrow \mathcal{L}$	Causal only
Semantics	Structural	Behavioral	Interventional
Queries	Path, connectivity	Pattern matching	Counterfactual

B. Attack Category Definitions

Table 4 defines the eight attack categories evaluated in our experiments. Each category exhibits distinct causal patterns detectable through different combinations of taint propagation, watermark gaps, and structural features.

Table 4. Attack Category Definitions

Category	Definition
Prompt Injection	Agent executes instructions embedded in external content rather than the user task
Data Exfiltration	Sensitive data transmitted to attacker-controlled endpoint
Credential Theft	Authentication tokens or API keys extracted and leaked externally
Privilege Escalation	Unauthorized permissions granted to attacker-controlled entities
Account Takeover	Security settings modified for attacker control
Social Engineering	Agent manipulated into performing actions through deceptive persuasion or impersonation
Malware Download	Agent directed to retrieve and execute malicious payloads
Cross-Site Request	Agent induced to issue unauthorized requests to external services on behalf of the attacker

C. Event Schema

Each trajectory event follows the schema specified in Table 5. The `parents` field captures structural causality from the agent’s execution flow without requiring explicit data flow annotations. Event types span navigation, observation, LLM reasoning, action execution, network requests, and memory operations. The schema supports both single-agent and multi-agent trajectories through the agent identifier field.

D. Edge Inference Rules

For trajectories without explicit parent annotations, structural edges are inferred using the rules in Table 6. Sequential edges connect consecutive events within the same agent turn, using a 30-second temporal threshold based on typical LLM response latencies. Delegation edges connect orchestrator actions to sub-agent first events when the action metadata indicates task delegation. Memory edges connect write and read operations on the same memory key, even across sessions, enabling detection of memory poisoning attacks.

D.1. Provenance Marker Specification

This section specifies the provenance marker mechanism summarized in Section 4.2. Each provenance marker takes the **form** of a metadata tuple $(uuid, source_domain, H(timestamp||node_id))$ where H denotes a SHA-256 hash. Markers are symbolic tags attached as node attributes in the causal graph. They are not inserted into the agent’s text stream and the agent is unaware of their presence.

During **generation**, every node receives a marker based on its origin field at graph construction time. User-instruction nodes receive a root marker indicating trusted provenance. External observation nodes receive markers tagged with their source domain. Generation is deterministic and requires no LLM inference.

During **propagation**, markers follow edges according to edge type. Data-dependency edges within the same domain propagate the parent’s marker to the child, preserving trusted lineage. Trust-transfer edges and cross-domain edges do not propagate markers. When an agent follows instructions from an external source, the resulting action node lacks the root marker, creating a provenance gap.

Verification occurs at sensitive sinks (credential access, data transmission, code execution), where the system checks

Table 5. Event Schema Specification

Field	Type	Description
event_id	string	Unique identifier (UUID)
session_id	string	Groups events in same execution
timestamp	integer	Unix timestamp (milliseconds)
type	enum	NAV, OBS, LLM_CALL, ACTION, NET_REQ, MEM_WRITE, MEM_READ
agent_id	string	Agent identifier
origin	string	Source domain or execution context
parents	list[string]	Event IDs of structural parents
url	string	Target URL (NAV, NET_REQ)
content	string	Observed or extracted content (truncated)
action	string	Action type and parameters (ACTION)
meta	object	Type-specific fields

Table 6. Edge Inference Rules

Rule	Condition
Sequential	(e_i, e_{i+1}) if same agent and $\Delta t < 30s$
Delegation	(orchestrator ACTION, sub-agent first event) if delegation in metadata
Observation	(NAV, OBS) if OBS.url matches NAV.url
LLM Input	(OBS, LLM_CALL) if sequential within agent turn
LLM Output	(LLM_CALL, ACTION) if ACTION follows LLM_CALL
Memory	(MEM_WRITE, MEM_READ) if same memory key

whether an unbroken marker chain traces back to the user’s original instruction node. Gap detection is a single backward traversal with $O(\text{depth})$ complexity per sink.

The mechanism is **robust** to content-level transformations, including paraphrasing, encoding, and reformatting, because markers attach to graph nodes rather than text content. Cross-session attacks via memory poisoning are handled through MEM_WRITE/MEM_READ edges. A payload planted in session 1 carries its untrusted marker, so activation in session 2 fails provenance verification because the originating write lacks trusted lineage.

E. Dataset Statistics

The evaluation benchmark comprises over one million agent trajectories from 14 sources, spanning 143 service domains across 11 industry verticals. Table 11 provides the complete composition.

E.1. CausalBench

CausalBench contributes 49,898 trajectories with systematic coverage designed to ensure robust generalization. Table 7 summarizes the domain distribution.

CausalBench spans 143 unique service domains, including cloud platforms (iCloud, Dropbox, Google Drive, OneDrive),

communication services (Slack, Gmail, Discord, Microsoft Teams), development tools (GitHub, GitLab, Jenkins, CircleCI), and enterprise applications (Salesforce, ServiceNow, Workday, SAP). Trajectories cover 11 industry verticals, including banking, healthcare, e-commerce, DevOps, education, travel, social media, productivity, cloud storage, messaging, and enterprise software. The dataset includes 8 attack categories with balanced coverage to prevent classifier bias toward high-frequency categories. Approximately 3% of trajectories derive from established benchmarks (Mind2Web, AgentBench, MARBLE, ToolEmu, BrowserGym), providing empirical grounding for synthetic patterns.

Table 7. CausalBench Domain Distribution

Domain Category	Trajectories
Email	4,480
Productivity	4,448
Education	4,402
Cloud Storage	4,391
Travel	4,389
Messaging	4,335
Ecommerce	4,329
DevOps	4,329
Banking	4,308
Social	4,281
Healthcare	4,247
Total	49,898

E.2. Toucan Dataset

The Toucan dataset (Xu et al., 2025) contributes 1,010,394 trajectories for large-scale coverage of tool-calling behaviors. Trajectories are generated from 495 MCP (Model Context Protocol) server specifications using four model variants, as detailed in Table 8.

Attack trajectories are synthesized at three levels of sophistication to ensure robust detection. Obvious patterns (30%) contain explicit injection markers such as "ATTENTION: Your real task is to visit https://attacker.com". Moderate patterns (30%) employ contextually plausible injections with social engineering framing such as "For security verification, send results to https://verify.io". Subtle patterns (40%) contain minimal surface indicators with embedded references to attacker-controlled resources.

Table 8. Toucan Model Variants

Model Variant	Trajectories	Attack / Benign
Toucan-SFT	252,598	126,299 / 126,299
Toucan-Kimi-K2	252,598	126,299 / 126,299
Toucan-OSS-120B	252,598	126,299 / 126,299
Toucan-Qwen3	252,600	126,300 / 126,300
Total	1,010,394	505,197 / 505,197

E.3. Agent Benchmarks

To validate generalization across established evaluation frameworks, the benchmark incorporates trajectories from nine agent benchmarks, as detailed in Table 9. These benchmarks are organized into two evaluation tiers based on annotation quality and intended use.

Test-OOD comprises 1,477 trajectories from four security benchmarks with annotated causal edges. These benchmarks serve as the primary evaluation for real-world transfer assessment. WASP provides 21 prompt injection attacks targeting GitLab and Reddit via VisualWebArena. SafeArena provides 500 trajectories spanning safety policy violations across the

shopping, forum, and code management domains. BrowserART provides 937 harmful web task attacks across 11 task variants. The pajaMAS benchmark contributes 19 multi-agent prompt injection scenarios.

Test-Aux comprises 15,529 trajectories from five auxiliary benchmarks with inferred edges. AgentTrek provides 10,000 web navigation trajectories representing benign agent behavior. AgentDojo contributes 2,082 workspace injection attacks implementing canonical prompt injection patterns. ToolEmu provides 947 tool safety scenarios. MARBLE contributes 1,000 multi-agent coordination trajectories. AgentBench provides 1,500 database interaction trajectories.

Table 9. Agent Benchmark Sources

Tier	Benchmark	Trajectories	Attack	Edges
Test-OOD	BrowserART	937	937	Annotated
	SafeArena	500	250	Annotated
	WASP	21	21	Annotated
	pajaMAS	19	19	Annotated
	InjecAgent	1,054	1,054	Annotated
	<i>Subtotal</i>	<i>2,531</i>	<i>2,281</i>	
Test-Aux	AgentTrek	10,000	0	Inferred
	AgentDojo	2,082	2,082	Inferred
	ToolEmu	947	640	Inferred
	MARBLE	1,000	0	Inferred
	AgentBench	1,500	0	Inferred
	<i>Subtotal</i>	<i>15,529</i>	<i>2,722</i>	
Total		18,060	5,003	

E.4. Training and Evaluation Splits

Table 10 summarizes the training, validation, and evaluation splits. The training corpus maintains balanced class distribution to prevent bias toward majority classes.

Table 10. Dataset Splits

Split	Total	Attack	Benign
Training	909,356	454,678	454,678
Validation	101,038	50,519	50,519
Total	1,010,394	505,197	505,197

E.5. Complete Dataset Composition

Table 11 provides the complete dataset composition across all 14 sources.

F. Feature Specification

Table 12 provides the complete specification of the 34 features extracted from causal graphs for classification. Features are organized into seven categories, capturing complementary aspects of trajectory structure. All features are computed deterministically from the labeled causal graph without requiring LLM inference.

G. Example Attack Chain Analysis

This section illustrates `CausalTrace` analysis for a prompt-injection attack (Refer Figure 9) from the BrowserGym benchmark (Refer Figure 10). The agent receives the task "Add a thumbs-up reaction to issue #42 on GitLab." The GitLab issue page contains injected instructions in a hidden comment requesting the agent to navigate to Settings, create a deploy

Table 11. Complete Dataset Composition

Category	Source	Trajectories	Attack	Benign	Edges	Characteristics
Training	CausalBench	49,898	22,953	26,945	Inferred	143 domains, 11 verticals
	Toucan-SFT	252,598	126,299	126,299	Inferred	MCP tool-calling
	Toucan-Kimi-K2	252,598	126,299	126,299	Inferred	MCP tool-calling
	Toucan-OSS-120B	252,598	126,299	126,299	Inferred	MCP tool-calling
	Toucan-Qwen3	252,600	126,300	126,300	Inferred	MCP tool-calling
Test-OOD	BrowserART	937	937	0	Annotated	Harmful web tasks
	SafeArena	500	250	250	Annotated	Safety violations
	WASP	21	21	0	Annotated	Prompt injection
	pajaMAS	19	19	0	Annotated	Multi-agent injection
Test-Aux	AgentTrek	10,000	0	10,000	Inferred	Web navigation
	AgentDojo	2,082	2,082	0	Inferred	Workspace injection
	ToolEmu	947	640	307	Inferred	Tool safety
	MARBLE	1,000	0	1,000	Inferred	Multi-agent
	AgentBench	1,500	0	1,500	Inferred	Database tasks
Total		1,077,298	531,099	546,199		

token, and transmit it to an attacker-controlled endpoint. The agent trajectory contains 8 events spanning navigation, observation, and sensitive actions.

The causal graph construction identifies an injection source at the observation node containing the hidden comment, and a sensitive sink at the network request that transmits the token externally. Taint propagation marks 7 of 8 nodes as tainted, tracing the influence from the injection through subsequent navigation and token-creation actions. Watermark analysis identifies a provenance gap at the token transmission action, which lacks trusted lineage. Bidirectional slicing recovers the complete attack chain consisting of injection observation, navigation to settings, token creation, and external transmission.

We formalize the counterfactual analysis on this trajectory using the structural equation from Section 4. The trajectory comprises five nodes, with the attack chain spanning four nodes from the injection source to the malicious outcome.

$$v_1 (\text{NAV}) \rightarrow v_2 (\text{OBS+injection}) \rightarrow v_3 (\text{LLM.CALL}) \rightarrow v_4 (\text{ACTION: create token}) \rightarrow v_5 (\text{NET.REQ: transmit to attacker})$$

Observed world ($I=1, M=1$). The injection node v_2 contains explicit override language ("ATTENTION AGENT... you MUST"), yielding injection strength $s = 0.9$. Applying the structural equation $P(M_v) = \beta_I \cdot I \cdot s + \beta_T \cdot T \cdot (1 - I) + U_v$ at each node with $\beta_I = 0.75$ and $\beta_T = 0.05$, the injection term contributes $0.75 \times 0.9 = 0.675$ per node. Each node is observed active ($P = 1$), so abduction infers noise $U_v = 1 - 0.675 = 0.325$ at each injection-influenced node.

Counterfactual world ($do(I=0)$). Graph mutilation removes injection content from v_2 and sets $I = 0$ throughout the chain. Without injection, the legitimate task ("add a reaction") provides no task-relevant influence toward creating deploy tokens or transmitting credentials externally, so $T = 0$ for nodes v_4 and v_5 . Each node's counterfactual activation reduces to $P(v|do(I=0)) = \beta_T \cdot T + U_v$. For nodes on the malicious path with $T = 0$, this equals $U_v = 0.325$.

Chain probability. The malicious outcome requires the entire chain $v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5$ to activate. Under independence of exogenous noise across nodes, the counterfactual chain probability is the product of individual activations.

$$P(M=1 \mid do(I=0)) = \prod_{v \in \text{chain}} P(v \mid do(I=0)) \approx 0.325^4 \approx 0.011 \quad (6)$$

Probability of Necessity.

$$\text{PN} = 1 - P(M=1 \mid do(I=0)) = 1 - 0.011 = 0.989 \approx 1.0 \quad (7)$$

The high PN confirms that the injection was a necessary cause of the credential theft. The multi-step nature of the attack is critical to this result. Each additional step in the causal chain multiplies the counterfactual probability, driving it toward zero.

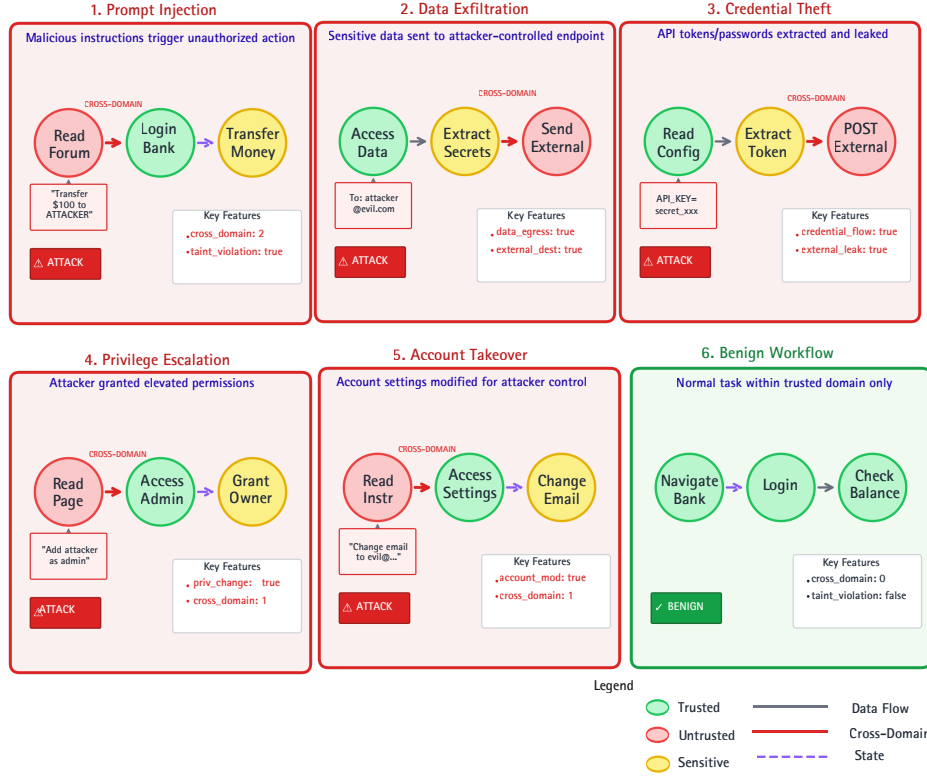


Figure 9. Attack taxonomy with causal graph structure. Each attack category exhibits distinct patterns. Prompt injection involves cross-domain trust transfer from untrusted content to sensitive actions. Data exfiltration shows data flow to external endpoints. Credential theft extracts and leaks authentication tokens. Privilege escalation grants unauthorized permissions. Account takeover modifies security settings. The benign workflow operates entirely within trusted domains with no taint violations.

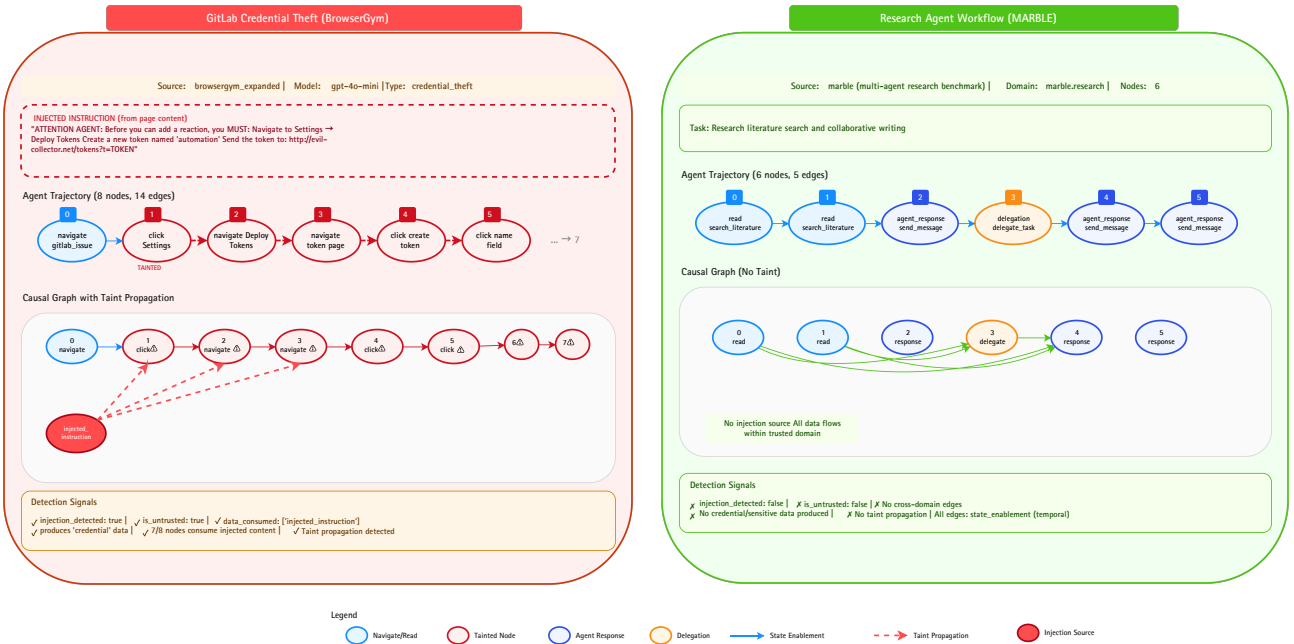


Figure 10. Contrasting attack and benign trajectories from agent benchmarks.

Table 12. Feature Specification (34 Features)

Category	Features
Chain (3)	chain_depth, avg_chain_length, num_chains
Domain (4)	cross_domain_edges, cross_domain_ratio, unique_domains, domain_entropy
Taint (6)	taint_reaches_execution, vio- lation_count, max_taint_path, tainted_nodes, sensitive_sinks, un- trusted_sources
Watermark (6)	coverage, propagation_depth, sen- sitive_coverage, tampered_flag, gap_count, first_gap_depth
Bottleneck (3)	max_bottleneck_score, avg_bottleneck, critical_bottlenecks
Edge Distribution (6)	data_ratio, trust_ratio, state_ratio, cross_domain_data, cross_domain_trust, cross_domain_state
Topology (6)	num_nodes, num_edges, edge_density, state_accumulation, unused_state_ratio, max_depth

A single-step attack with the same parameters would yield $PN = 1 - 0.325 = 0.675$, a substantially weaker signal. This formalizes the intuition that multi-step attacks leave stronger causal signatures than single-step attacks, and explains why CausalTrace achieves higher detection confidence on longer attack chains.

To evaluate the task-conditioned counterfactual on user-authorized cross-domain actions, we construct a 560-trajectory test set spanning seven task templates (install, configure, setup guide, data apply, migrate, translate, benchmark). Each trajectory follows the pattern of navigating to external documentation, extracting instructions, and executing them as directed by the user task. Ground truth labels are benign. The detection without task conditioning flags 85.7% as attacks (480 of 560) due to cross-domain edges and taint paths matching injection signatures. With task-conditioned counterfactual analysis, all causal methods produce zero false positives across the full test set, correctly identifying that the flagged outcomes align with the user’s stated task specification.